OpenCode Setup Reference

Everything this configuration can do, how to invoke it, and when it applies

`Machine`	macOS, Apple silicon
`OpenCode`	2.0.25
`Config root`	~/.config/opencode -> ~/dotfiles/opencode (git, main)
`Loaded from`	opencode.jsonc, cli.json, commands/, agents/, plugins/, skills/, AGENTS.md
`Generated`	23 October 2026

This describes the setup as it exists on this machine, not OpenCode in general. Where the installed build differs from the published documentation, this document follows the build and says so, because several documented keys are not present in 2.0.25. Section 15 shows how to verify that a configuration change actually took effect.

Sections 1 to 12 are reference material: what each part of the configuration does and how to change it. Section 13 gives the two workflows you use most. Sections 14 and 15 cover what this build cannot do, and how to keep the setup healthy.

Contents

1. How this setup is wired . . . 4

Precedence . . . 4

Applying changes . . . 4

2. Agents . . . 4

Reaching an agent . . . 5

When to use which . . . 5

3. Slash commands . . . 5

Mechanics worth knowing . . . 5

Adding one . . . 6

When to use them . . . 6

4. Models . . . 6

Variants, and the temperature trap . . . 6

Credentials . . . 7

5. Skills . . . 7

Access control . . . 7

Adding a skill . . . 7

The alias plugin . . . 8

6. Plugins . . . 8

Managing plugins . . . 8

Enabling and disabling . . . 8

Terminal-only plugins . . . 8

7. Permissions and policies . . . 8

What you currently allow and deny . . . 9

Adding a hard denial . . . 9

Actions that permissions and policies can name . . . 9

8. References . . . 10

What they actually do . . . 10

Adding one . . . 10

When to use one . . . 10

9. Formatter . . . 10

Enabling Python formatting . . . 11

The markdown caveat . . . 11

10. Language servers . . . 11

11. Web search . . . 12

12. Terminal settings . . . 12

Everything else is available but unset . . . 12

13. Recipes . . . 12

A paper question . . . 12

A code change . . . 13

Choosing where a new capability goes . . . 13
MCP servers . . . 13
14. Known limits on 2.0.25 . . . 13
15. Maintenance and troubleshooting . . . 14
Inspecting the live state . . . 14
Logs . . . 14
Verifying a change actually took effect . . . 14
Upkeep . . . 14
Appendix: what was verified on this machine . . . 15

1. How this setup is wired

Your configuration directory is a symlink into your dotfiles repository, so the two paths are the same files and everything is version controlled.

```
~/.config/opencode   ->   ~/dotfiles/opencode      (git, branch main)
```

OpenCode splits configuration across two files that do not overlap. Putting a setting in the wrong one fails silently, because neither file is validated against the other.

File	Controls	Scope
`opencode.jsonc`	Server behaviour: models, agents, permissions, policies, skills, plugins, LSP, formatter, references, web search	Global
`cli.json`	Terminal client only: theme, alerts, tabs, diffs, keybinds, mini	Global, this machine
`commands/*.md`	Reusable slash commands	Global
`agents/*.md`	Custom agents	Global
`plugins/*.js`	Local plugins, auto-discovered	Global
`skills/<id>`	Skills you author, one SKILL.md each	Global
`AGENTS.md`	Instructions injected into every agent	Global

Precedence

Settings merge from lowest to highest authority. Files that do not conflict are all preserved.

```
1  system      /Library/Application Support/opencode
2  global      ~/.config/opencode/opencode.jsonc      <- yours
3  project     opencode.jsonc, outer dirs beat inner
4  project     .opencode/opencode.jsonc, overrides all direct files
```

Policies reverse this. They are evaluated with the broadest configuration last, so a repository you clone cannot re-enable something you deny globally. Permissions do not have that property: a project can re-allow anything the global file only denies via `permissions`.

Applying changes

- Files under watched config directories reload on their own, mid-session. Yours are watched, including the symlink target.
- `opencode reload` forces a reload without restarting the server.
- `opencode service restart` when the service is stuck. This interrupts running sessions.
- `opencode debug config` prints the decoded configuration the server actually holds. Use it instead of trusting the raw file.

> A setting can be accepted, decoded, and still be inert if the key does not exist in your installed build. Section 15 covers how to catch that.

2. Agents

Agents are named profiles: a system prompt, a model preference, permissions, and a mode. You have two custom ones and the standard built-ins.

Agent	Mode	What it is for
build	primary	The default. Full tool access, asks before reading `.env` files
plan	primary	Explores and plans without editing project files. Denies edits, so it is safe for review work

Agent	Mode	What it is for
general	subagent	Research and multi-step work in a child session. Cannot launch further subagents (depth is 1)
explore	subagent	Search and read only. Cheapest way to map a codebase
chat	primary	Yours. Web search answers, no filesystem, no skills. Bounded to 2 tool steps
paper	primary	Yours. Zotero and vault questions, grounded in PDF text and your notes
compaction, title, summary	hidden	Internal maintenance. Not selectable

Reaching an agent

- **Tab** cycles the primary agents in the TUI.
- A slash command can switch agent for that invocation with `agent:` in its frontmatter. `/review` does this, pinning `plan`.
- Ask the primary agent to delegate: "use the explore subagent to find ...". Only subagent-mode agents can be launched that way.
- A background child session is available through a command with `subagent: true`.

When to use which

- **build** for anything that changes files.
- **plan** before a change you are unsure about, and for reviews, because it cannot edit.
- **explore** to answer "where is X" without spending context on a full agent run.
- **general** for independent research that would otherwise pollute your main context.
- **chat** for a quick sourced answer with no filesystem access at all.
- **paper** for anything touching papers, references, or literature notes - it carries the vault layout and citekey conventions that no other agent has.

3. Slash commands

Three commands exist, all global, all verified registered on this machine.

Command	Runs as	What it does
`/init`	build	Reads the repo, then writes or refreshes its AGENTS.md
`/review`	plan	Reads the full working diff and reports findings in severity order. Cannot edit
`/commit`	build	Stages explicit paths and commits in your Conventional Commits style

Mechanics worth knowing

Syntax	Behaviour
$ARGUMENTS	The whole argument string as typed
$1, $2, ...	Positional arguments. Quoted groups count as one. The highest placeholder consumes the rest
no placeholder	Non-empty arguments are appended after a blank line
!`cmd`	Runs the command and injects its output before submitting. **Runs outside the normal tool permission flow** - only trust command files you wrote
agent:	Switches the session agent for that run
model:	provider/model or provider/model#variant
subagent: true	Runs in a background child session; the parent stays free

Adding one

Drop a markdown file in `~/.config/opencode/commands/`. The body is the prompt; frontmatter carries metadata. Nested directories become namespaced commands, so `team/review.md` is `/team/review`. A project file at `.opencode/commands/<name>.md` overrides the global one of the same name. JSON definitions under a `commands` key work too, but files are easier to edit.

```
---
description: Review uncommitted changes for correctness and missing tests
agent: plan
---

Review the current uncommitted changes in this repository.

$ARGUMENTS
```

When to use them

- Run **/init** once when you start working in an unfamiliar repository. It is the fastest way to give every later agent the context it needs.
- Run **/review** before you commit, or before opening a PR, when you want a second opinion that cannot quietly fix things itself.
- Run **/commit** when you are happy with the diff and want the commit message to match your repo's existing convention.

4. Models

You have one configured provider, Paratera, exposed as an OpenAI-compatible endpoint.

```
"providers": {
  "paratera": {
    "package": "aisdk:@ai-sdk/openai-compatible",
    "settings": { "baseURL": "https://llmapi.paratera.com" },
    ...models...
  }
}
```

Model id	Shown as
`DeepSeek-V4.1-Flash`	DeepSeek V4.1 Flash (the default)
`DeepSeek-V4-Pro-0813`	DeepSeek V4 Pro 0813
`GLM-5.3`	GLM 5.3
`GLM-5.3-Flash`	GLM 5.3 Flash
`GLM-4-Flash`	GLM 4 Flash (free)
`GLM-4-FlashX`	GLM 4 FlashX
`Qwen3.8-Max`	Qwen 3.8 Max
`Kimi-K3`	Kimi K3

Variants, and the temperature trap

Variants are the only way to set per-agent request options in V2. An agent pins the model as `provider/model#variant`. Agent-level `temperature` is accepted but never sent, so it does nothing.

```
"DeepSeek-V4.1-Flash": {
  "variants": [{ "id": "chat", "body": { "temperature": 0.3 } }]
}
```

That variant is what the `chat` agent uses. The root `model` setting cannot carry a variant.

Credentials

Stored accounts, as reported by `opencode auth list`:

Provider	Method	State
Paratera	API key	stored
OpenRouter	API key, plus OPENROUTER_API_KEY in the environment	stored
Tavily	API key, used by web search	stored
Z.AI Coding Plan	API key	stored

- Switch model in the TUI with the model picker, or per run with `opencode run --model provider/model`.
- Add a model by adding an entry under `providers.paratera.models`.
- The `opencode` provider (OpenCode Zen) is denied by policy, so it never appears in the catalogue.

5. Skills

Skills are instruction bundles, not tools. Each is a directory with a SKILL.md whose description is what the model matches against. Yours come from four places, and 75 are advertised in total.

Source	Count	Notes
`~/.opencode-skills/nature-skills/skills`	23	nature-* academic writing, review, figures, citations
`~/.opencode-skills/anthropic-skills/skills`	19	documents, PDFs, spreadsheets, decks, design, skill authoring
`~/.config/opencode/skills`	1	zotero-cli, authored for your paper workflow
`superpowers plugin`	30	15 skills, registered twice: bare id and superpowers:<id> alias
`built in`	2	opencode (OpenCode's own questions) and report

Access control

Skills can be denied per agent through the `skill` permission action, and this is exactly what you do:

```
// global: nobody gets the academic pack by default
"permissions": [{ "action": "skill", "resource": "nature-*", "effect": "deny" }]
```

The `paper` agent re-allows `nature-*` and `zotero-cli`. Because global rules are evaluated first and agent rules are appended afterwards, that re-allow works. The net effect is that the 23 nature skills are available only inside the paper agent, and every other agent's prompt stays free of them.

> Do not move that denial into `policies`. A policy can only be lifted by broader configuration, never by an agent rule, so the paper agent would lose its academic pack.

Adding a skill

1. Create `<dir>/<skill-id>/SKILL.md` with YAML frontmatter containing at least a `name` and a `description`.
2. Either put it under `.opencode/skills/` for one project, or add its parent directory to the global `skills` array.
3. Write the description as the trigger condition - that sentence is the only thing the model sees when deciding whether to load the skill.

The alias plugin

Superpowers registers its skills under bare directory names but refers to them internally as `superpowers:brainstorming`. OpenCode resolves skill ids by exact match, so those references would fail. Your local plugin `plugins/superpowers-skill-aliases.js` registers a hidden alias for each one. Hidden means no description, so the aliases cost no prompt tokens. The plugin is a no-op if upstream ever fixes the naming.

6. Plugins

Plugins extend OpenCode itself: new tools, hooks, transforms, agents, and commands. You have one package plugin and one local one.

Plugin	Kind	Purpose
`superpowers@git+https://github.com/obra/superpowers.git`	package (git)	Registers ~15 methodology skills and their aliases. Needs OpenCode 2.0.4 or newer
`plugins/superpowers-skill-aliases.js`	local file	Adds superpowers:<id> aliases so the plugin's own cross-references resolve

Local files under `~/.config/opencode/plugins/` are discovered automatically and do not need listing in the `plugins` array.

Managing plugins

```
opencode plugin list                # configured and installed
opencode plugin list --builtin      # include what ships with OpenCode
opencode plugin add         # install and add to global config
opencode plugin check               # which have updates
opencode plugin update              # update all outdated
opencode plugin remove      # uninstall
```

Accepts npm names with versions, tags or ranges, and git specs including `github:owner/repo` and private repositories over SSH. Exact versions and full commit hashes stay pinned.

Enabling and disabling

- Entries are processed in order. Prefix with `-` to disable, use `*` for all, or `id.*` for an id prefix. A later entry can re-enable.
- Two built-ins refuse removal because they enforce policy: `opencode.config.policy` and `opencode.provider.opencode`.
- There are about 60 built-in server plugins covering providers, config loading, MCP, worktrees, VCS and prompting. `plugin list --builtin` enumerates them.

Terminal-only plugins

Plugins that extend the TUI are configured separately under `plugins` in `cli.json`, and stay active even when the TUI is attached to a remote server.

7. Permissions and policies

These two look similar and behave differently. Getting them confused is the most common way to be surprised by this setup.

	permissions	policies
Declared in	opencode.jsonc as `permissions`, or per agent under `agents.<id>.permissions`	`experimental.policies` on your build
Decisions	allow, deny, ask	allow or deny only
Prompts you	Yes, on ask	Never

	permissions	policies
Ordering	Last match wins. Lower-priority config first, then global, then agent rules appended	Last match wins. Broadest configuration evaluated last
Can a project or agent lift it	Yes, a later rule can re-allow	No. Only managed or Console configuration
Beats a saved Allow always	A configured deny does	Yes, always

In one line: use permissions when you want to be asked or want a rule an agent can refine; use policies for things that must never happen, in any agent, in any repository.

What you currently allow and deny

- **Baseline for every agent**, from OpenCode itself: allow everything, but ask before reading `*.env` and `*.env.*` files. So today most operations run without a prompt.
- **Global rule**: deny the `nature-*` skills.
- **paper agent**: re-allows `nature-*` and `zotero-cli`, allows read/grep/glob/webfetch, allows any `zotero-cli` command, and asks before other shell commands and before any edit.
- **chat agent**: deny everything, then allow web search and web fetch.
- **Policies**: deny the `opencode` provider, and hard-deny reading or editing anywhere under `~/.ssh` or `~/.gnupg`, plus reading `~/.aws/credentials`.

Nothing currently prompts before touching paths outside the project, because the default for `external_directory` is allow. To restore a prompt, add the rule already noted in a comment in your config:

```
{ "action": "external_directory", "resource": "*", "effect": "ask" }
```

Adding a hard denial

Policy resources are the permission action and value joined by a colon, with the same whole-value wildcards. Put broad statements first; the last match wins, so exceptions go after.

```
"experimental": {
  "policies": [
    { "action": "tool.use", "resource": "shell:sudo *", "effect": "deny" },
    { "action": "tool.use", "resource": "read:*/.ssh/*", "effect": "deny" }
  ]
}
```

Pattern	Blocks
`shell:*`	Every shell command
`read:*/.ssh/*`	Reading SSH material
`edit:*/.env`	Writing environment files
`external_directory:*`	Any access outside the project
`subagent:general`	Launching one specific subagent
`github_delete_repository:*`	One MCP tool, by server and tool name
`*`	Everything not already denied

Actions that permissions and policies can name

Action	Resource it matches
`read`	Internal path, or the absolute path when external

Action	Resource it matches
`edit`	Target of edit, write and patch
`glob, grep`	The pattern or the regular expression, not the search path
`shell`	Each command the scanner produces
`subagent`	Target agent id
`skill`	Skill id
`webfetch, websearch`	URL, or the query string
`external_directory`	Canonical external directory boundary
`execute`	Code Mode; denying it removes the tools reached only through it
`<server>_<tool>`	One MCP tool

> An `ask` is not a guarantee. If you approve with Allow always, OpenCode saves a durable project-scoped allow rule. Only a `deny`, in permissions or policies, overrides that.

8. References

References make directories outside the project available to agents as named aliases. You have two.

Alias	Path	Why
`vault`	`~/vaults/climate-crisis`	Research vault; literature notes live under research-papers-<flask>/notes/<citekey>.md
`dotfiles`	`~/dotfiles`	This configuration plus nvim, zsh and tmux

What they actually do

- A reference with a `description` is advertised to agents in their instructions, so they know the path exists and what it is for. Without a description it is still attachable by the client but not advertised.
- A client attaches a reference by alias, which contributes a non-recursive listing of the root. Content below the root is reached with normal reads.
- **References grant no permissions.** Reads and edits still go through the read, edit and external_directory rules, so this is about discoverability, not access.
- A local reference needs a leading `.`, `/` or `~`. A bare value like `docs` is read as a git repository name.
- A reference can also be a remote repository - `"effect": "Effect-TS/effect"` clones and refreshes it into the data directory.

Adding one

```
"references": {
  "sdk": {
    "path": "~/work/shared-sdk",
    "description": "Use for SDK implementation details"
  }
}
```

When to use one

Add a reference when a directory outside the project comes up often enough that agents should know its name, rather than being handed an absolute path each time or guessing. If you are already working inside a directory, a reference to it adds nothing.

9. Formatter

Formatting is enabled. After the write, edit or patch tools change a file, OpenCode runs the first matching built-in formatter whose executable is actually present.

Formatter	Extensions	State on this machine
`gofmt`	.go	active, gofmt is installed
`rustfmt`	.rs	active, rustfmt is installed
`ruff`	.py, .pyi	inactive, ruff is not installed
`uv`	.py, .pyi	inactive, uv 0.5.9 predates uv format
`prettier`	js, ts, css, json, yaml, md, ...	inactive, needs a prettier dependency in the project
`biome, oxfmt`	js, ts, css, json, ...	inactive, need project config

So in practice this currently formats Go and Rust only. Your Python files are untouched, which is why enabling it did not disturb your workflows.

Enabling Python formatting

The built-in entry needs a `ruff` command on PATH. Without one, switch to the object form - which still enables all the built-ins - and point the entry at a runner:

```
"formatter": {
  "ruff": {
    "command": ["uvx", "ruff", "format", "$FILE"],
    "extensions": [".py", ".pyi"]
  }
}
```

The markdown caveat

Prettier also covers `.md`, which is the only way formatting could reach your Zotero literature notes. It cannot happen today: prettier only runs inside a project that depends on it, and your vault is not an npm project. If you ever add prettier to a repository that also holds markdown you care about, disable it for that extension:

```
"formatter": { "prettier": { "extensions": [".js", ".ts", ".json"] } }
```

> Disable a single built-in with `"prettier": { "disabled": true }` while leaving the others enabled. Set the whole key to `false` to turn formatting off.

10. Language servers

One language server is configured: basedpyright for Python, running in workspace diagnostic mode.

```
"lsp": {
  "basedpyright": {
    "command": ["basedpyright-langserver", "--stdio"],
    "extensions": [".py", ".pyi"],
    "initialization": { "diagnosticMode": "workspace" }
  }
}
```

- LSP diagnostics are what let an agent see a type error it just introduced, so this is worth having for any language you edit often.
- Add another server the same way: a `command` array and the `extensions` it owns.
- Workspace diagnostic mode checks the whole workspace rather than open files, which is slower on large projects but catches breakage outside the file being edited.

11. Web search

Web search is wired to Tavily, so the websearch tool answers with a real integration rather than scraping.

```
"websearch": { "provider": "tavily" }
```

- Needs `TAVILY_API_KEY` in the environment or a signed-in account. Yours is stored, so no environment variable is required.
- The free tier is 1,000 credits per month, one credit per basic search.
- `"provider": "random"` picks any available provider automatically. Setting the key to `false` removes the tool entirely.
- Gated by the `websearch` permission action, which is how the `chat` agent keeps it while losing everything else.

12. Terminal settings

`cli.json` controls the terminal client only. It has no project-local form and cannot be overridden per repository. Your current contents are minimal:

```
{
  "theme": { "name": "system" },
  "attention": { "notifications": true, "sound": true }
}
```

`attention` gives you a desktop notification and a sound when a run finishes, asks a question, or needs a permission decision - the behaviour the other agent harnesses have by default.

Everything else is available but unset

Group	Settings	Worth knowing
`attention`	volume, sound_pack, per-event sounds	Override sounds for default, question, permission, error, done, subagent_done
`session`	sidebar, scrollbar, thinking, grouping, image_preview, tps, markdown, new_location, permissions	tps shows tokens per second; permissions autoaccept skips every prompt
`diffs`	source, wrap, tree, single, view	source accepts branch, committed, working or turn. turn shows only what the last turn changed
`input`	mouse, scroll.speed, scroll.acceleration, prompt.editor, prompt.paste, prompt.image_preview	prompt.editor includes your active editor file as context
`tabs`	mode, scope, layout, indicators	layout vertical puts the tab strip in a sidebar
`keybinds`	leader, any command id, leader.timeout	Set a leader key, then bind `<leader>` sequences
`terminal`	title, copy	copy controls select-to-copy
`mini`	thinking, tools, shell_output, turn_summary, footer, splash, work_spinner, mono, replay	Applies to the lightweight opencode mini interface
`debug`	devtools, timing, turn_tokens	turn_tokens verbose adds tool call inputs

Every one of these reloads while the TUI is running; no restart needed. Unknown settings are rejected rather than ignored, so a typo is visible. The same values are reachable interactively with **Ctrl+P** then **Open settings**.

13. Recipes

A paper question

1. Switch to the **paper** agent. It is the only agent allowed to see the nature skills and the Zotero skill.

2. Identify the paper. A note filename is the citekey; a citekey resolves to a Zotero item key.
3. Read the literature note first. Your summary and coloured annotations are treated as your position, not as paper content.
4. Read narrowly: outline the paper, then a page range aimed at the question.
5. Expect page citations, and provenance labelled as paper text versus your annotations versus inference.

```
zotero-cli --json search --mode citekey  --detail keys_only
zotero-cli outline
zotero-cli read  --start-page 12 --end-page 18
zotero-cli search --mode semantic "soil carbon feedback"
```

A code change

1. **/init** once, in a repository that has no AGENTS.md.
2. Ask the **plan** agent to work out the approach if the change is not obvious. It cannot edit, so it cannot get ahead of you.
3. Let **build** make the change.
4. **/review** before committing; it reads the whole diff and reports by severity.
5. **/commit** to stage explicit paths and write a conventional message.

Choosing where a new capability goes

You want	Use	Because
Reusable instructions for a kind of task	a skill	Model-selected by description, costs nothing until used
A repeatable prompt you invoke on purpose	a command	Explicit trigger, can pin agent and model
New tools, hooks, or TUI behaviour	a plugin	The only extension point with executable code
A whole different persona or access level	an agent	Own prompt, model and permissions
A directory outside the project to be known by name	a reference	Advertised to agents, grants no permissions
Something that must never happen	a policy	Cannot be lifted by a project or an agent

MCP servers

None are configured, deliberately. MCP tool schemas are sent with every request, and the Zotero MCP server costs roughly 13k tokens of context per call, which is why `zotero-cli` exists instead. If you do want one:

```
opencode mcp add context7 --url https://mcp.context7.com/mcp
opencode mcp add everything --global -- npx -y @modelcontextprotocol/server-everything
opencode mcp list
```

Remote servers use OAuth by default; if one needs signing in, use `/mcps` in the TUI rather than the shell. MCP tools are reached through Code Mode by default, which keeps their schemas out of your main context. Set `codemode: false` on a server to list its tools directly.

14. Known limits on 2.0.25

Your installed build is 2.0.25 and the published documentation describes newer builds. These are the differences that matter, all confirmed against the installed binary.

Documented feature	On 2.0.25	Workaround
`experimental.todo`, giving a todowrite tool	Does not exist. The experimental schema is exactly portable_shell_scanner, subagent_depth and policies	Keep planning in a markdown file, or upgrade OpenCode

Documented feature	On 2.0.25	Workaround
Top-level `policies` key	Ignored without any warning; the file still decodes	Use `experimental.policies`, which is what your config now does
Agent-level `temperature`	Accepted but never sent with the request	Define a model variant and pin `provider/model#variant`

> `experimental.subagent_depth` defaults to 1, so a subagent cannot launch its own subagents. Raise it only if you actually want nested delegation. `experimental.todo` is the one item from your upgrade list that this build cannot deliver; `opencode upgrade` is the fix.

15. Maintenance and troubleshooting

Inspecting the live state

Command	Tells you
`opencode debug config`	The decoded configuration the server actually holds, after validation and normalisation
`opencode debug agents`	Every agent, including hidden ones
`opencode debug paths`	Where data, config and cache live
`opencode api get /api/agent`	Agents with their effective permission lists
`opencode api get /api/command`	Every registered slash command
`opencode models`	The full model catalogue as the server sees it
`opencode service status`	Whether the background service is healthy

Logs

```
~/.local/share/opencode/log/opencode.log

# client startup vs sessions, providers, plugins, tools
grep 'role=cli'    ~/.local/share/opencode/log/opencode.log
grep 'role=server' ~/.local/share/opencode/log/opencode.log
```

Run a single reproduction with `OPENCODE_LOG_LEVEL=DEBUG` when the normal log is not enough.

Verifying a change actually took effect

A config edit can decode cleanly and still do nothing, as the policies placement did. Check the decode, then check a behaviour. The pattern that worked here:

1. Decode check: `opencode debug config` and look for your key. Absence means it was dropped; the key is not in this build's schema.
2. Behaviour probe: pick something the setting must change. A denied provider leaves the catalogue, so `opencode models | grep -c '^opencode/'` going from 9 to 0 proved the policy was live.
3. If a documented key is missing from the decode, stop trusting the documentation for your build and read the schema out of the binary instead.

Upkeep

```
opencode reload            # re-read config, keep sessions
opencode service restart   # when the service is wedged
opencode plugin update     # update package plugins
opencode upgrade           # install a newer OpenCode
```

Plugin changes under watched config directories reload automatically. Exact npm versions and full git commit hashes stay pinned, so nothing upgrades behind your back unless you asked for it.

Appendix: what was verified on this machine

The claims above were checked against the running installation rather than taken from documentation. The checks that carry the most weight:

Claim	How it was confirmed
Policies are enforced	With the provider deny in place, the OpenCode models in the catalogue went from 9 to 0 while the Paratera count stayed at 8
References reach agents	A reference added mid-session was injected into the running session's context without a restart
Commands are registered	`/api/command` lists init, review and commit
Skill count	`/api/skill` advertises 75 skills across the four sources
Formatter is enabled	The flag appears in the decoded configuration
Attention keys exist	`attention` and `sound_pack` are present in the 2.0.25 binary, so these settings are not silently ignored
Agent temperature does nothing	The field decodes but never reaches the request body
There is no todowrite tool	The experimental schema is exactly portable_shell_scanner, subagent_depth and policies
Top-level policies are ignored	A deny placed there had no behavioural effect until it was moved under experimental.policies

> Re-run the decode check after any edit to this file. Silent drops are the failure mode of this configuration format, not errors.